CANNBot开发进阶
Ascend C 算子开发实操

作者：吴世真

时间：2026.04.14

为什么需要 Agent Team 开发算子?

Ascend C直调开发方式介绍：Ascend C直调支持在同一源码中编写Host与Device Kernel代码，并通过异构编译一次完成构建与联调；开发者可直接发起Kernel调用，减少工程拆分和样板代码，提升开发效率与可维护性。

传统开发痛点

API查不到找不对	不知道用什么API，查文档耗时且容易选错。
开发周期长	从设计→编码→调试→精度达标，人工开发以「周」计
代码审查靠人走读	质量依赖个人经验，问题易遗漏

Agent Team 怎么解决?

知识沉淀为 Skills	1022 份 API 文档 + 最佳实践 + 精度标准，Agent 自动检索，不靠人记
流程固化为 Pipeline	7 步标准流程 + 门禁机制，每步产出文件做交接，不跳步不遗漏
质量内建为角色	Architect/Developer/Reviewer各司其职

课程目标

Ascend C开发实操	一句话需求 → 小时级交付可运行算子
理解Agent team协同模式	多个 AI Agent 分工合作
精度 / 性能问题排查	Agent 自动定位症状根因并修复

前置知识

算子开发基础	掌握Ascend C算子开发基础
VSCode操作	熟悉VSCode基本使用

传统： 人工开发 · 天级周期 · 经验依赖 → **Agent Team：** 自动协同 · 小时级交付 · 知识内建 · 质量可控

目录

Part 1 Agent Team 介绍

Part 2 Agent开发流程介绍

Part 3 实操演示

Ascend C Kernel 直调算子 — Agent Team

四大 Agent 协同 · 职责分离 · Skills 驱动

TeamLead

全局调度 · 仲裁决策

接收用户需求，执行环境检查，
按流程调度子 Agent，仲裁争议，
输出最终报告。不编写任何代码。

Skills

- ascendc-env-check
- ascendc-docs-search
- ascendc-precision-debug

Architect

需求分析 · 架构设计

Tiling 设计、API 验证、
多核/UB 切分策略，
输出 DESIGN.md + PLAN.md。

Skills

- ascendc-tiling-design
- ascendc-npu-arch
- ascendc-api-best-practices
- ops-precision-standard
- ascendc-docs-search

Developer

编码实现 · 编译测试

基于模板创建工程，增量编译，
功能测试 (L0-L2)，性能采集，
更新 PLAN.md，编写 README。

Skills

- ascendc-direct-invoke-template
- ascendc-api-best-practices
- ascendc-docs-search
- ascendc-precision-debug
- ascendc-runtime-debug
- ops-profiling
- ascendc-env-check

Reviewer

独立审查 · 质量评分

独立编译验证，精度/性能复核，
输出 REVIEW.md (PASS/FAIL)。

Skills

- ascendc-code-review
- ascendc-docs-search
- ops-profiling
- ops-precision-standard
- ascendc-api-best-practices

协作机制：Architect 专注设计 | Developer 专注实现 | Reviewer 独立评审 | 文件驱动交接

CANN

目录

Part 1 Agent Team 介绍

Part 2 Agent开发流程介绍

Part 3 实操演示

开发流程 Pipeline

7 步流水线 · 设计串讲 · 最多 3 轮修复 · 性能验收

1 环境检查

TeamLead

NPU 设备检测
CANN 环境校验

产出

environment.json

→

2 架构设计

Architect

Tiling 策略
API 选型验证

产出
DESIGN.md
PLAN.md

→

2.1 设计串讲

Dev ↔ Arch

Developer 评审设计
Architect 回应
TeamLead 仲裁

产出

WALKTHROUGH.md

→

3 编码开发

Developer

模板创建工程
增量编译 A→D
功能测试 L0-L2

产出
*.asc CMake
gen_golden.py

4 代码审查

Reviewer

独立编译验证
7 维度百分制评分
精度复测

产出

REVIEW.md

→

5 修复循环

Dev → Rev

Developer 修复问题
Reviewer 复审
最多 3 轮

产出

更新代码 + REVIEW.md

→

6 性能验收

Developer

msprof 性能采集
瓶颈分析优化

产出

perf/round_N/
summary.txt

→

7 最终报告

TeamLead

汇总评分
代码路径 · 性能结论

产出

Final Report

CANN

目录

Part 1 Agent Team 介绍

Part 2 Agent开发流程介绍

Part 3 实操演示

实操演示

算子需求

开发一个 **Softmax算子**，对输入张量沿指定轴做 softmax 变换，将 logits 转换为概率分布（将一组实数归一化为概率分布，输出值均在 (0, 1) 且和为 1）

数学表达式

$$Softmax(x_i) = \frac{e^{x_i - \max(x)}}{\sum_{j=1}^{N} e^{x_j - \max(x)}}$$

预计耗时

1小时

技术要求

数据类型	FP32
计算精度	1e-5
支持的Shape	[128, 64, 32]

开发流程预览

1. 启动CANNBot并输入需求
2. 自动设计方案与代码生成
3. 代码审查与质量保障
4. 编译运行与验证

Thank you.

群聊：CANNBot与Skills SIG交流

该二维码7天内(4月17日前)有效，重新进入将更新

扫码加入微信交流群

https://gitcode.com/cann

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

CANN